



DOCUMENTO DE PLANIFICACIÓN ÁGIL

Sprint 1 — Front-End

Proyecto: Integrador2_Proyecto — ERP de Reclutamiento

Semana: Planificación y Front-End — Semana 3

Stack tecnológico: Front-End: Next.js · Back-End: FastAPI (Sprint 2) · Base de datos: PostgreSQL

Herramienta de gestión: Github

Elaborado por: Grupo 01

1. PLAN DE GESTIÓN DE RIESGOS

Elemento	Decisión del equipo
Objetivo y alcance	Proteger el Sprint Goal vigente y la calidad del incremento; cubre entorno, repositorio, prototipo y código.
Método	Identificación grupal guiada por la RBS, análisis cualitativo con matriz 5×5, tratamiento con dueño y disparador, seguimiento en cada reunión del equipo.
Categorías (RBS)	Negocio y valor · Alcance y requerimientos · Calendario y equipo · Arquitectura y tecnología · Datos e integración · Seguridad y privacidad y accesibilidad · Calidad y pruebas · Infraestructura y despliegue · Operación y soporte.
Escalas	Probabilidad e impacto de 1 a 5. Exposición = $P \times I$.
Umbrales	1-4 Bajo (monitorear) · 5-9 Moderado (dueño y respuesta proporcional) · 10-16 Alto (acción priorizada, revisión semanal) · 17-25 Crítico (actuar antes de continuar, escalar de inmediato).
Roles	Dueño del riesgo: jefatura de proyecto. Dueño de acción: quien ejecuta la tarea. Aprobador: Product Owner.
Cadencia	Planificación del Sprint, cada daily breve y antes de cada hito (APF1/APF2/APF3).
Registro y evidencia	El registro de la sección 2 de este documento, más un issue con la etiqueta riesgo para cada riesgo Alto o Crítico (plantilla .github/ISSUE_TEMPLATE/riesgo.md).
Reserva	Un día de trabajo por integrante reservado antes del cierre del Sprint.
Comunicación	Los riesgos Altos y Críticos se reportan al Product Owner / docente en la sustentación semanal.

2. Registro de riesgos priorizado

ID	Riesgo (causa → evento → impacto)	Cat.	P	I	P×I	Nivel
R-02	Debido al número de historias pendientes, el equipo podría intentar construir demasiado a la vez y comprometer el Sprint Goal.	Calendario y equipo	4	4	16	Alto
R-05	Debido a controles personalizados, usuarios de teclado podrían no percibir el foco o activar el resultado.	UX y accesibilidad	3	4	12	Alto
R-07	Debido a que el Back-End se divide en varios microservicios en lugar de un monolito, podría aumentar la sobrecarga operativa y retrasar la integración del Sprint 2.	Arquitectura y tecnología	3	4	12	Alto

R-08	Debido a que cada microservicio expone su propia API y esquema de datos, podrían surgir contratos inconsistentes si no se documentan con OpenAPI desde el inicio.	Datos e integración	3	4	12	Alto
R-03	Debido al uso de ejemplos reales durante las pruebas, podrían incorporarse datos personales o secretos al repositorio.	Seguridad y privacidad	2	5	10	Alto
R-09	El workflow CI Frontend (.github/workflows/ci.yml) declaraba working-directory: frontend y cache-dependency-path: frontend/package-lock.json,	Infraestructura y despliegue	3	4	12	Alto
R-04	Debido a anchos fijos en el listado, el diseño podría perder información en pantallas pequeñas.	UX y accesibilidad	3	3	9	Moderado
R-06	Debido a versiones distintas de Node.js entre integrantes, la instalación podría fallar y retrasar la integración.	Infraestructura y despliegue	2	3	6	Moderado

3. Mapa de calor (matriz 5×5)

P \ I	1 Insig.	2 Menor	3 Moderado	4 Mayor	5 Severo
5	5	10	15	20	25
4	4	8	12	16 R-02	20
3	3	6	9 R-04	12 R-05, R-07, R-08, R-09	15
2	2	4	6 R-06	8	10 R-03
1	1	2	3	4	5

4. Respuestas, disparadores y riesgo residual

ID	Estrategia y acción	Disparador / contingencia	Residual	Evidencia
R-02	Mitigar: priorizar el backlog cada semana en el tablero,	Disparador: más de 2 historias sin avanzar de	Moderado (P2/I4=8)	Captura del tablero GitHub

ID	Estrategia y acción	Disparador / contingencia	Residual	Evidencia
	límite de 1 ítem «In progress» por persona, revisar el Sprint Goal en cada daily.	columna en una semana. Contingencia: mover historias de baja prioridad a Sprint 2 y comunicarlo al Product Owner/docente.		Projects con el límite WIP aplicado.
R-05	Mitigar: usar elementos nativos (button, input, select) con estado :focus-visible y anotar el orden de foco en cada componente de Postulantes, Anuncios y Empresas.	Disparador: la navegación por teclado (Tab/Shift+Tab) no permite activar un control. Contingencia: añadir tabIndex y atributos aria-explicitos hasta corregir el componente.	Bajo (P2/I3=6)	Registro de prueba de teclado (Actividad 10 de esta semana).
R-07	Aceptar con monitoreo: es una decisión ya tomada en el ADR-002; se prepara infra/compose.yaml con un servicio por dominio antes del Sprint 2.	Disparador: hacia el Sprint 2 no se logran levantar los 3 servicios + gateway de forma reproducible (docker compose config falla). Contingencia: consolidar temporalmente 2+ microservicios sin perder la separación por dominio en el código.	Alto hasta iniciar Sprint 2 (se revisa entonces)	ADR-002, sección «Criterio de revisión».
R-08	Mitigar: documentar cada microservicio con OpenAPI/Swagger desde el inicio del Sprint 2; mientras tanto, congelar los contratos en features/*/types/ como referencia provisional.	Disparador: un cambio de tipo no tiene equivalente documentado para el futuro Back-End. Contingencia: revisión conjunta Front-End/Back-End antes de integrar.	Moderado (P2/I4=8) hasta integración real	Carpeta features/*/types/ versionada y enlazada al ADR-002.
R-03	Mitigar: usar exclusivamente datos sintéticos (mockPostulantes.ts, mock-anuncios.ts, mock_empresas.ts) y exigir el ítem «No contiene secretos ni datos personales» del checklist de PR.	Disparador: un PR incluye un dato que parece real o un archivo .env sin ignorar. Contingencia: revertir el commit, rotar credenciales y limpiar el historial según CONTRIBUTING.md.	Bajo (P1/I5=5)	Checklist de PR marcado y revisión cruzada registrada.
R-09	Mitigar: reemplazar el único working-directory: frontend por una matriz (strategy.matrix.app: [ERP, ANUNCIOS]) que ejecuta	Disparador: el job falla o se salta pasos en el próximo PR. Contingencia: si una de las dos apps no debe construirse todavía, quitarla	Moderado hasta que el equipo aplique el ci.yml	ci.yml corregido entregado al equipo; log de ejecución del workflow en la

ID	Estrategia y acción	Disparador / contingencia	Residual	Evidencia
	build-and-lint por separado en frontend/ERP y frontend/ANUNCIOS.	temporalmente de la matriz en vez de volver a un solo working-directory.	corregido y lo confirme en un PR real (sección 8.1)	pestaña Actions del primer PR posterior al cambio.

Patrones aplicados

Patrón / práctica	Propósito	Dónde se aplica en el incremento
Diseño por componentes	Dividir la interfaz por responsabilidades reutilizables.	PostulantesSearch, PostulantesFilters, PostulantesTable.
Contenedor y presentación	Separar orquestación de estado del renderizado.	usePostulantesList.ts (contenedor) vs. PostulantesTable.tsx (presentacional).
Estado explícito	Evitar booleanos contradictorios.	idle / loading / success / empty / error en usePostulantesList.
Composición	Construir la vista a partir de piezas pequeñas.	app/(dashboard)/postulantes/page.tsx compone búsqueda + filtros + tabla.
Mobile-first / responsive	Adaptar prioridad y espacio sin duplicar interfaces.	Utilidades responsive de Tailwind en PostulantesTable / PostulantesList.
Mejora progresiva semántica	Partir de controles nativos y agregar comportamiento.	input, select y button nativos en filtros y búsqueda.